

Punycode Email Identity-Confusion Testbed

Punycode Email Identity-Confusion Testbed - Author not stated, Voorivex.

- Published: date not stated
- Original: <<https://github.com/VoorivexTeam/white-box-challenges/tree/4824536ed0db7f747f9c1b2a311a3f9e22e719c4/punycode>>
- Preserved from: <https://github.com/VoorivexTeam/white-box-challenges/tree/4824536ed0db7f747f9c1b2a311a3f9e22e719c4/punycode> (manual-import) on 2026-09-13
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Punycode Email Identity-Confusion Testbed

Authors:

Source: <https://github.com/VoorivexTeam/white-box-challenges/tree/4824536ed0db7f747f9c1b2a311a3f9e22e719c4/punycode>

Preservation note: These are the complete text contents of the reviewed files from this pinned revision. File contents were checked against their Git blob hashes. This document preserves source text for reading; it is not a runnable repository copy.

[punycode/README.md](#)

NahamCon - Puny-Code: 0-Click Account Takeover

[punycode/app.js](#)

```
const express = require('express');
const session = require('express-session');
const passport = require('passport');
const path = require('path');
require('dotenv').config();
require('./config/passport');

const app = express();

// View engine setup
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));
app.use(express.static(path.join(__dirname, 'public')));
app.use(express.urlencoded({ extended: true }));

// Express session
app.use(session({
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false
}));

app.use(passport.initialize());
app.use(passport.session());

// Routes
app.use('/', require('./routes/index'));
app.use('/', require('./routes/auth'));
app.use('/', require('./routes/gitlab'));

app.listen(3000, () => console.log('App running on http://localhost:3000'));
```

punycode/config/db.js

```
require('dotenv').config();
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  user: process.env.DB_USER,
  host: process.env.DB_HOST,
  database: process.env.DB_DATABASE,
  password: process.env.DB_PASSWORD,
  port: process.env.DB_PORT,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});

module.exports = pool;
```

punycode/config/mail.js

```
require('dotenv').config();
const nodemailer = require('nodemailer');

const transporter = nodemailer.createTransport({
  service: "Gmail",
  host: "smtp.gmail.com",
  port: 465,
  secure: true,
  auth: {
    user: process.env.MAIL_USER,
    pass: process.env.MAIL_PASS,
  },
});

module.exports = transporter;
```

punycode/config/passport.js

```

require('dotenv').config();
const passport = require('passport');
const GitLabStrategy = require('passport-gitlab2').Strategy;
const pool = require('./db');

passport.use(new GitLabStrategy({
  clientID: process.env.GITLAB_CLIENT_ID,
  clientSecret: process.env.GITLAB_CLIENT_SECRET,
  callbackURL: process.env.CALLBACK_URL
}), async (accessToken, refreshToken, profile, done) => {
  try {
    const email = profile.emails?.[0]?.value || '';
    if (!email) return done(new Error('No email provided by GitLab'));

    const conn = await pool.getConnection();
    const [rows] = await conn.execute('SELECT * FROM users WHERE email = ?', [email]);

    conn.release();

    if (rows.length === 0) {
      return done(null, false, { message: 'No such user in the database, please register first.' });
    }

    const user = rows[0];
    return done(null, {
      id: user.id,
      name: user.name,
      email: user.email,
      created_at: user.created_at
    });
  } catch (err) {
    return done(err);
  }
});

passport.serializeUser((user, done) => done(null, user));
passport.deserializeUser((obj, done) => done(null, obj));

```

punycode/config/schema.sql

```
CREATE TABLE IF NOT EXISTS users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  email VARCHAR(255) NOT NULL UNIQUE,  
  password VARCHAR(255) NOT NULL,  
  reset_token VARCHAR(255),  
  reset_expires DATETIME,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

punycode/middleware/auth.js

```
function ensureAuthenticated(req, res, next) {  
  if (req.isAuthenticated()) return next();  
  res.redirect('/');  
}  
  
module.exports = { ensureAuthenticated };
```

punycode/routes/auth.js

```

const express = require('express');
const crypto = require('crypto');
const bcrypt = require('bcryptjs');
const pool = require('../config/db');
const router = express.Router();
const transporter = require('../config/mail');

// Forgot password form
router.get('/forgot-password', (req, res) => {
  res.render('forgot-password');
});

// Handle forgot password request
router.post('/forgot-password', async (req, res) => {
  const { email } = req.body;

  const token = crypto.randomBytes(32).toString('hex');
  const expires = new Date(Date.now() + 3600000); // Token expires in 1 hour

  // Get a connection from the database pool
  const conn = await pool.getConnection();

  try {
    // Update the user record with the reset token and expiration time
    await conn.execute(
      'UPDATE users SET reset_token = ?, reset_expires = ? WHERE email = ?',
      [token, expires, email]
    );
  } catch (err) {
    console.error('Error updating user record:', err);
    return res.status(500).send('Internal Server Error');
  } finally {
    conn.release();
  }

  // Prepare email options
  const mailOptions = {
    from: 'hhhcv9@gmail.com', // Sender's email
    to: email, // Recipient's email
    subject: 'Password Reset', // Email subject
    text: `Reset link: http://localhost:3000/reset-password/${token}`, // Reset link
  };

  // Send the password reset email

```

```

transporter.sendMail(mailOptions, (err, info) => {
  if (err) {
    console.error('Error sending email:', err);
  } else {
    console.log('Email sent:', info.response);
  }
});

// Respond to the client indicating that the reset link has been sent
res.render('forgot-password', { message: 'Reset password link sent!' });
});

// Show form to reset password
router.get('/reset-password/:token', async (req, res) => {
  const { token } = req.params;

  const conn = await pool.getConnection();
  const [rows] = await conn.execute(
    'SELECT * FROM users WHERE reset_token = ? AND reset_expires > NOW()',
    [token]
  );
  conn.release();

  if (rows.length === 0) {
    return res.send('Reset link is invalid or expired.');
```

```

    return res.send('Invalid or expired token.');
```

```

  }

  const hashedPassword = await bcrypt.hash(password, 10);

  await conn.execute(
    'UPDATE users SET password = ?, reset_token = NULL, reset_expires = NULL WHERE reset_token = ?',
    [hashedPassword, token]
  );
  conn.release();

  res.redirect('/?message=Password reset successful. Please log in.');
```

```

});

module.exports = router;
```

punycode/routes/gitlab.js

```

const express = require('express');
const passport = require('passport');
const router = express.Router();

// GitLab OAuth
router.get('/auth/gitlab', passport.authenticate('gitlab', { scope: ['read_user'] }));

router.get('/auth/gitlab/callback',
  passport.authenticate('gitlab', { failureRedirect: '/?error=User not found, please register first' }),
  (req, res) => {
    if (req.user) {
      return res.redirect('/profile');
    }
  }
);

module.exports = router;
```

punycode/routes/index.js

```
const express = require('express');
const router = express.Router();
const pool = require('../config/db');
const bcrypt = require('bcryptjs');
const { ensureAuthenticated } = require('../middleware/auth');

router.get('/', (req, res) => {
  const { error = null, message = null } = req.query;

  res.render('index', { error, message });
});

router.get('/signup', (req, res) => {
  const { error = null, message = null } = req.query;

  res.render('signup', { error, message });
});

router.post('/signup', async (req, res) => {
  const { name, email, password } = req.body;

  const conn = await pool.getConnection();
  const [rows] = await conn.execute('SELECT * FROM users WHERE email = ?', [email]);

  if (rows.length > 0) {
    conn.release();
    return res.redirect('/signup?error=Email already exists');
  }

  const hashedPassword = await bcrypt.hash(password, 10);
  await conn.execute('INSERT INTO users (name, email, password) VALUES (?, ?, ?)', [name, email, hashedPassword]);

  conn.release();
  res.redirect('/?message=Account created successfully');
});

router.post('/login', async (req, res) => {
  const { email, password } = req.body;

  const conn = await pool.getConnection();
  const [rows] = await conn.execute('SELECT * FROM users WHERE email = ?', [email]);

  if (rows.length === 0) {
    conn.release();
  }
});
```

```
    return res.redirect('/?error=User not found');
  }

  const user = rows[0];
  const isMatch = await bcrypt.compare(password, user.password);
  conn.release();

  if (!isMatch) {
    return res.redirect('/?error=Invalid password');
  }

  req.login(user, err => {
    if (err) return res.redirect('/?error=Login failed');
    res.redirect('/profile');
  });
});

router.get('/profile', ensureAuthenticated, (req, res) => {
  res.render('profile', { user: req.user });
});

router.get('/logout', (req, res, next) => {
  req.logout(err => {
    if (err) return next(err);
    res.redirect('/');
  });
});

module.exports = router;
```